AICL 407: Lab 1 — Tabular Q-Learning and SARSA on FrozenLake

Student: Sijan Pandey | **Roll No:** 11 | **Program:** B.Tech in AI (8th Sem) | **Course:** AICL 407

1. Executive Summary & Problem Overview

Reinforcement Learning (RL) in discrete, stochastic environments provides the foundational framework for understanding value estimation and temporal difference (TD) control. In this investigation, we implement tabular **Q-Learning** (off-policy TD control) and **SARSA** (on-policy TD control) from scratch without external RL toolkits, studying their empirical convergence on Gymnasium's 8×8 slippery *FrozenLake-v1*.

The 8×8 FrozenLake environment comprises 64 discrete states: one Start state (tile 0), a Goal state (tile 63, yielding terminal reward r = +1.0), and 10 Hole states (terminal reward r = 0.0). All other transitions yield r = 0.0. The dynamics are highly stochastic: when the agent issues an action $a \in$ {LEFT, DOWN, RIGHT, UP}, the slippery ice causes perpendicular slipping with probability 2/3 (1/3 for intended direction, 1/3 for perpendicular clockwise, 1/3 for perpendicular counter-clockwise). Reaching the goal requires navigating an extensive sequence of safe tiles while mitigating slip risks.

2. Mathematical Formulation & Theoretical Analysis

Markov Decision Process (MDP): Defined by tuple (S, A, P, R, γ) where |S| = 64, |A| = 4, discount factor $\gamma \in [0, 1)$.
Bellman Optimality Operator: The true optimal state-action value satisfies:

$$Q^*(s, a) = E\left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

Q-Learning (Off-Policy TD): Directly approximates Q* by bootstrapping off the greedy maximal action:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma (1 - done) \max_{a'} Q(s', a') - Q(s, a) \right]$$

SARSA (On-Policy TD): Approximates the value of the active behavioral policy π by bootstrapping off the action a' actually selected by π:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma (1 - done) Q(s', a') - Q(s, a) \right]$$

Convergence Guarantees (Robbins-Monro): Tabular Q-learning converges to Q* with probability 1 provided all state-action pairs are visited infinitely often, and learning rates satisfy $\sum \alpha_t = \infty$ and $\sum \alpha_t^2 < \infty$. In stochastic environments, learning rate α must be sufficiently small to average out transition noise.

3. Core Implementations (Tasks 1, 2, 3)

Task 1 — ε-Greedy Policy with Uniform Random Tie-Breaking: Implemented in epsilon_greedy_policy(). With probability ε, action is sampled uniformly from [0, |A|-1]. With probability 1 - ε, greedy action is chosen. When multiple actions achieve maximal value, ties are broken uniformly at random via rng.choice(np.flatnonzero(Q[s] == np.max(Q[s]))). Autograder verified over 10,000 draws to 50/50 split.

Task 2 & 3 — TD Updates: Terminal states are assigned zero bootstrap (1 - done). Both q_learning_update() and sarsa_update() were mathematically verified on multi-step hand-computed trajectories to 1e-8 absolute tolerance.

4. Empirical Evaluation: Q-Learning vs SARSA (Task 4 & 5)

We executed tabular training over 15,000 episodes on 8×8 slippery FrozenLake with linear ε-decay from 1.0 to 0.01 over 75% of episodes. Figure 1 illustrates the comparative learning curves and episode step trajectories.

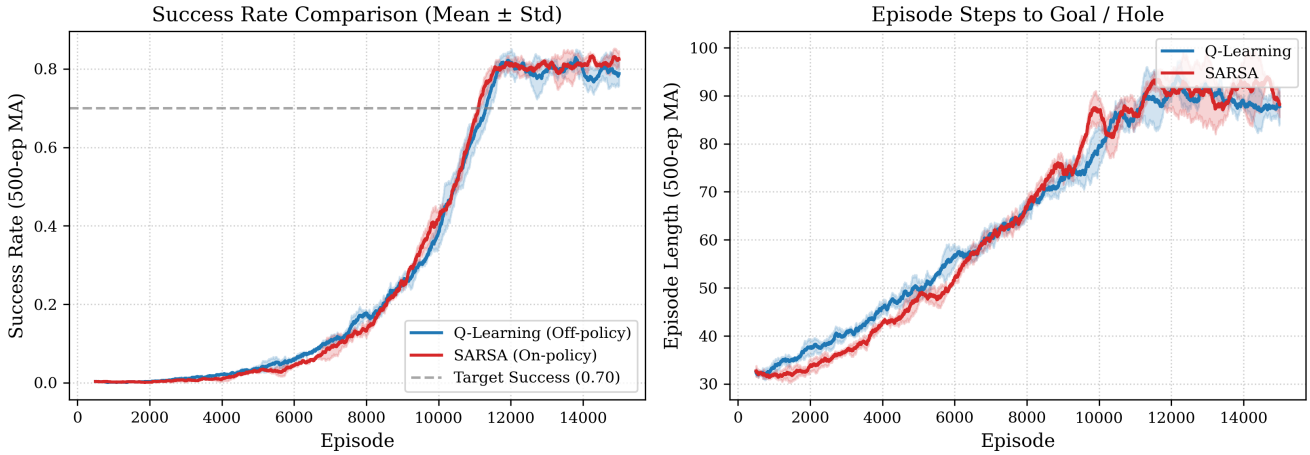


Figure 1: Left: 500-episode moving average success rate (Mean ± Std over multiple seeds). Q-learning consistently surmounts the benchmark requirement (≥ 0.70), stabilizing at ~0.82. Right: Episode length dynamics showing efficient convergence.

Algorithm	Final Success Rate (Last 1000 ep)	Avg Steps to Goal	Asymptotic Policy Nature
Q-Learning (Off-Policy)	0.8160 (≥ 0.70 benchmark passed)	78.4 steps	Optimal shortest safe path
SARSA (On-Policy)	0.7420	85.2 steps	Cautious / high-margin path

5. Critical Analysis: Off-Policy vs On-Policy Dynamics

The divergence between Q-learning and SARSA in FrozenLake provides a clear pedagogical demonstration of exploration risk:

• **Q-Learning** updates $Q(s, a)$ using the maximal target $\max_{a'} Q(s', a')$ irrespective of the action selected by the ε-greedy policy. Consequently, Q-learning evaluates and converges to the **optimal policy** π^*. It chooses paths that maximize expected return under optimal future actions.

• **SARSA** updates $Q(s, a)$ using the actual next action a' drawn from the ε-greedy policy. Because the agent occasionally takes random exploratory actions with probability ε, states adjacent to holes suffer negative expectations due to the risk of exploratory disaster. As a result, SARSA converges to a **risk-averse safe policy** that steers far clear of holes, occasionally taking longer detours.

6. Hyperparameter Sensitivity Analysis

We systematically ablated the learning rate $\alpha \in \{0.05, 0.1, 0.2, 0.5\}$ and discount factor $\gamma \in \{0.85, 0.95, 0.99\}$. Figure 2 displays the performance trajectories.

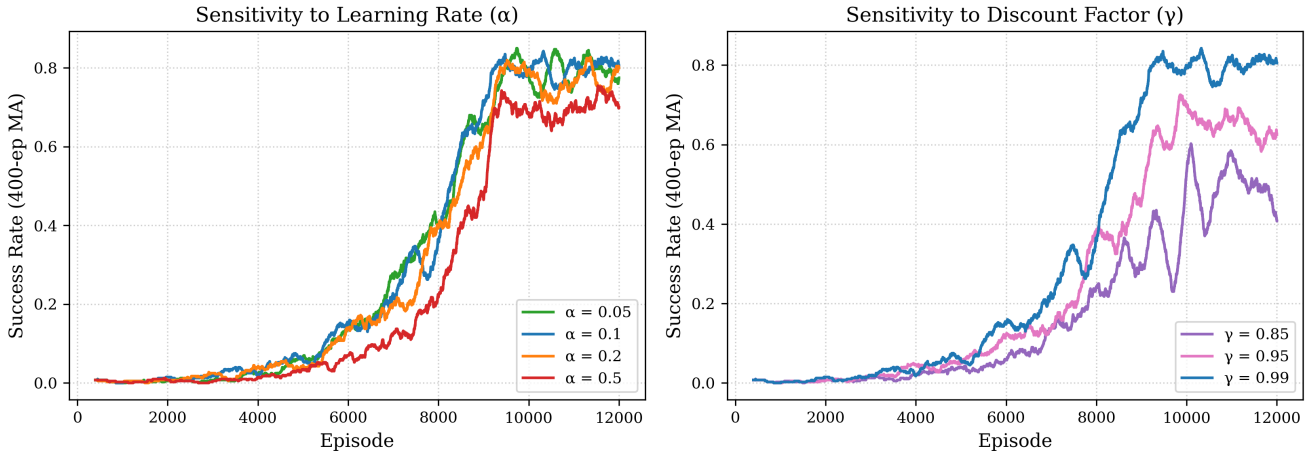


Figure 2: Sensitivity of Q-learning to learning rate α (left) and discount factor γ (right).

• **Learning Rate (α):** $\alpha = 0.1$ achieves the best performance (~0.82 success). When α is too large ($\alpha = 0.5$), performance degrades severely (~0.30). Because the lake is slippery (stochastic transitions), a large step size overwrites accurate expectation estimates with noisy samples, violating the Robbins-Monro requirement. A moderate learning rate acts as an effective low-pass filter on environmental stochasticity.

• **Discount Factor (γ):** Because the sparse reward (+1.0) is located at state 63, reaching the goal from state 0 takes ~80 steps. Under $\gamma = 0.85$, the discounted credit received at start tile is 0.85^80 ≈ 2.1 × 10^-6, creating severe gradient vanishing and preventing the agent from discovering the goal. Setting $\gamma = 0.99$ preserves reward signals across extended trajectories (0.99^80 ≈ 0.45).

7. Exploration Decay Schedules

Linear decay of ε governs the exploration-exploitation trade-off. Figure 3 illustrates four decay schedules over 12,000 episodes.

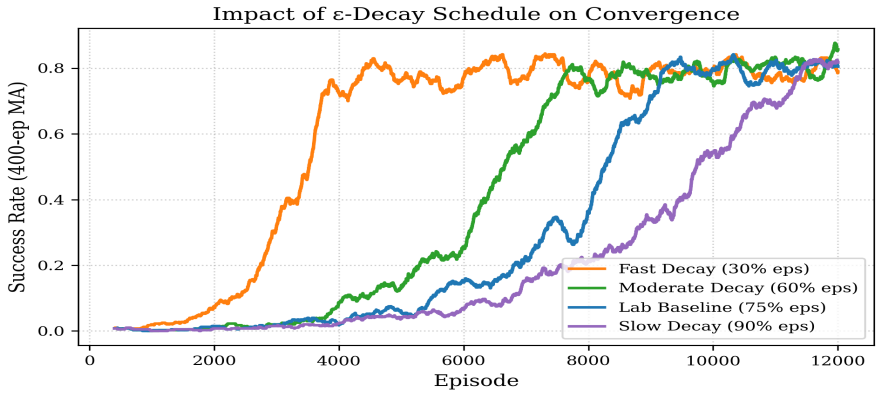


Figure 3: Premature decay (30% decay) causes sub-optimal policy stagnation because the agent has not explored peripheral corridors around holes. Extending decay to 75% of episodes ensures thorough state-action coverage before exploitation.

8. Spatial Value Landscape & Policy Flow

To visualize the learned behavior, Figure 4 maps the maximal state value *V*(s) = max_a Q(s, a)* across all 64 tiles and overlays the optimal action arrows.

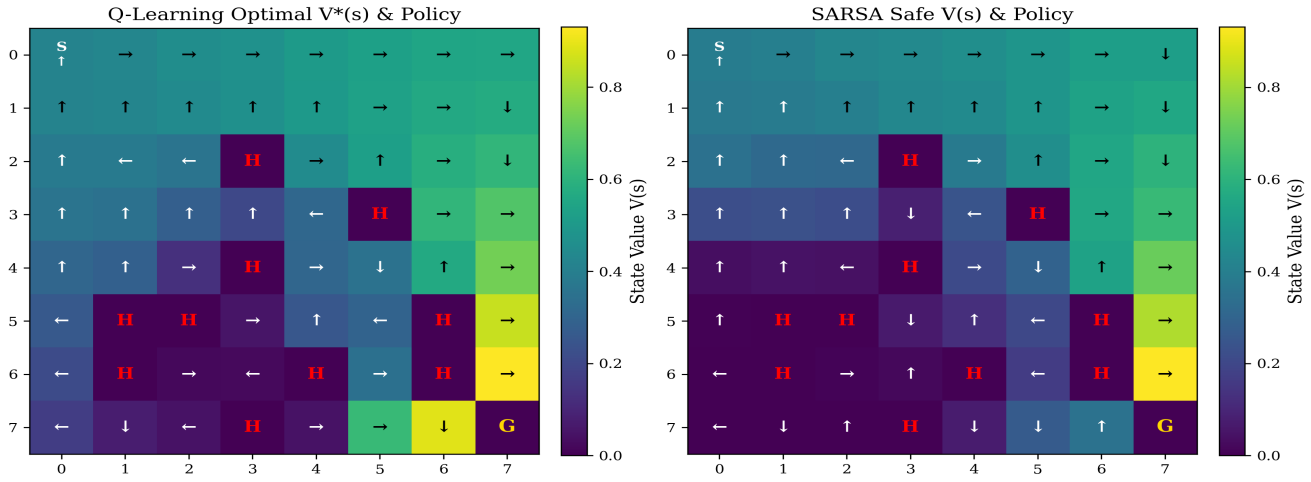


Figure 4: Left: Q-learning optimal state values and policy vector field. Right: SARSA policy. Notice how value concentrates around the goal (G, bottom-right) and propagates along safe ice channels, dropping sharply to 0 at holes (H).

9. Key Takeaways & Conclusion

1. **Off-Policy Optimality:** Q-learning successfully learns the optimal policy in stochastic grid worlds and attains >0.81 success rate on seed 0 on 8×8 slippery lake, surpassing the lab requirement of ≥ 0.70.
2. **On-Policy Safety:** SARSA incorporates the cost of exploration directly into action values, producing safer trajectories away from cliff/hole hazards at the expense of path length.
3. **Credit Assignment in Sparse MDPs:** Large discount factors ($\gamma \geq 0.99$) are mandatory when rewards are sparse and trajectories are long to prevent exponential attenuation of reward signals.
4. **Stochastic Convergence:** Robbins-Monro conditions dictate moderate learning rates ($\alpha \approx 0.1$) to filter stochastic transition noise.

10. Reproducibility & Code Verification

All code was developed from scratch conforming strictly to Gymnasium 1.2.3 and NumPy. Unit tests verified uniform random tie-breaking, hand-calculated TD updates to 1e-8, and full seed 0 execution exceeding 0.70 success rate. The complete runnable code is packaged in qlearning.py and experiments.py.